



شماره دانشجویی:

۸۱۰۱۰۱۴۹۲



تمرین شماره ۲

تئوری زنجیره‌های بلوکی
نام و نام خانوادگی: علیرضا کریمی

سوال ۱

(a)

همانطور که می‌دانیم، ما به کمک *privatekey* خود، *message* خود را امضا می‌کنیم و این امضا از دو عدد (r, s) تشکیل شده است. آنگاه فرد می‌تواند به کمک این امضا و *publickey* ما، اطمینان حاصل کند که این امضا معتبر است و برای ماست. نکته‌ای که مهم است، این است که امضای $(r, n - s)$ نیز معتبر است. [۱] n همان درجه *ellipticcurve* ما است. یعنی ما می‌توانیم n عدد مختلف روی *curve* خود داشته باشیم. با اینکه امضای متفاوتی است اما ثابت می‌کند که ما آن را امضا کردیم.

حال نکته اینجاست که از آنجایی که دو امضای معتبر داریم که هر دو اعتبار دارند، اگر یک بار تراکنش را با (r, s) امضا کنیم و بار دیگر با $(r, n - s)$ (که از نظر ریاضی امضایی معتبر است ولی شکل متفاوتی دارد)، آنگاه هش تراکنش‌ها متفاوت خواهد بود، با اینکه محتوای آن‌ها یکسان است. در نتیجه دو *txid* (منظورم هش تراکنش است) مختلف ایجاد می‌شود و می‌توان آن را برای حمله‌ی *double spend* به کار برد.

(اگر یکی از این تراکنش‌ها تأیید شود، هش آن سکه به‌عنوان خرج‌شده علامت می‌خورد، اما نسخه‌ی دیگر (با امضای متفاوت) هنوز در شبکه به‌عنوان سکه‌ی خرج‌نشده شناخته می‌شود.) این از مشکلات *ECDSA* است که *malleable* است. به عبارت دیگر، امضای آن قابل تعویض است. برای جلوگیری از آن، کافی‌ست که *sign* را خارج از هش قرار دهیم.

(b)

حتی اگر امضا از نظر ریاضی یکتا باشد، می‌توان با تغییر فرمت باینری یا متنی آن بدون تغییر مقدار واقعی امضا □ نسخه‌ی دیگری تولید کرد که هش متفاوتی دارد.

مثلاً اگر سمت چپ عدد امضا، صفر قرار داشته باشد، آن‌ها را حذف کنیم یا به سمت چپ آن صفر اضافه کنیم. در این حالت، با اینکه مقدار امضا تغییر نکرده، اما هش آن، به کلی تغییر خواهد کرد چون هش به تک‌تک حرف‌ها و کاراکترها

حساس است. پس دوباره می‌توانیم هش‌های متفاوتی ایجاد کنیم. همانطوری که گفتیم، راه‌حل آن، خارج کردن امضا از هش است. ابتدا هش می‌گیریم و بعد آن را امضا می‌کنیم. (درواقع راه‌حل آن، همان کاری است که در *SegWit* انجام شده است: امضا را از داده‌ای که برای محاسبه‌ی *txid* استفاده می‌شود جدا می‌کنیم، تا تغییر در امضا باعث تغییر شناسه‌ی تراکنش نشود.)

سوال ۲

(a)

کافی‌ست که *ScriptSig* ما به این صورت باشد:

$\langle \text{"Password"} \rangle$

در این حالت، یعنی ما *password* خود را در استک *push* می‌کنیم.

طبق *ScriptPubKey* ابتدا از آیتم سر استک که همین پسورد تازه اضافه‌شده ما است، $SHA - 256$ می‌گیریم. بعد از آن، عبارت هش خط دوم بر روی استک *push* می‌شود. بعد با $OP - EQUAL$ ، دو عضو سر استک را برمی‌داریم و چک می‌کنیم که با هم برابر باشند. اگر برابر بودند، ۱ و در غیر اینصورت صفر را در استک *push* می‌کنیم. بعد از آن، تنها در استک ما، یک عدد است. اگر ۱ بود، پسورد ما پذیرفته خواهد شد و تراکنش معتبر خواهد بود.

(b)

فرض کنید گذرواژه شامل کاراکترهای زیر باشد:

حروف کوچک انگلیسی $(a - z)$: ۲۶ حالت

حروف بزرگ انگلیسی $(A - Z)$: ۲۶ حالت

ارقام $(0 - 9)$: ۱۰ حالت

پس مجموع کاراکترهای ممکن برای هر موقعیت برابر است با:

$$26 + 26 + 10 = 62$$

برای یک گذرواژه ۸ کاراکتری، فضای جستجو برابر است با:

$$62^8 = 218,340,105,584,896 \approx 2.18 \times 10^{14}$$

فرض کنید یک کارت گرافیک مدرن می‌تواند حدود 10^9 هش $SHA-256$ در هر ثانیه محاسبه کند.

حداکثر زمان لازم برای شکستن گذرواژه (بدترین حالت):

$$\frac{62^8}{10^9} \approx 218,340 \text{ ثانیه} \approx 2.5 \text{ روز}$$

در صورت استفاده از تکنولوژی‌هایی مانند پردازش ابری یا دستگاه‌های خاص (ASIC) این زمان می‌تواند به چند ساعت یا حتی چند دقیقه کاهش یابد.

تازه باید در نظر داشته باشیم که محاسبات ما در $base 62$ بود و مثلاً در آدرس‌های بیت‌کوین از $base 58$ استفاده می‌شود و شکستن آن در این حالت به مراتب راحت‌تر است

(c)

با اضافه شدن ۱۲ کاراکتر به پسورد، مدت زمان شکستن رمز به اندازه‌ی 62^{12} برابر افزایش پیدا می‌کند. این یعنی شکستن چنین پسوردی از طریق *BruteForce* میلیون‌ها سال طول خواهد کشید، و در عمل غیرممکن خواهد بود. بنابراین، یک پسورد ۲۰ کاراکتری می‌تواند از نظر حمله‌ی جست‌وجوی فراگیر، تقریباً غیرقابل شکستن باشد. با این حال، هنوز ریسک‌های امنیتی دیگری وجود دارند. حتی اگر خطر استفاده از *RainbowTable* یا جداول هش معکوس شده را نادیده بگیریم، باز هم تهدیدهای دیگری باقی می‌ماند.

فرض کنید یک حمله‌ی زمانی (*TimingAttack*) اتفاق بیفتد. یعنی نیلوفر، پسورد را از طریق *ScriptSig* در شبکه پخش می‌کند. حالا، قبل از آنکه این تراکنش توسط ماینرها در بلاک قرار بگیرد، یک نود یا ماینر دیگر که سرعت بالاتری دارد، می‌تواند پسورد را از شبکه خوانده، آن را با هش موجود در *ScriptPubKey* مقایسه کند و اگر معتبر بود، با همان پسورد و همان *UTXO*، یک تراکنش جدید بسازد که بیت‌کوین‌ها را به آدرس خودش ارسال می‌کند. اگر آن فرد کارمزد (*Fee*) بالاتری برای تراکنش خود تعیین کند، ماینرها به احتمال زیاد تراکنش او را زودتر از تراکنش نیلوفر در بلاک بگنجانند، و در نتیجه، بیت‌کوین‌ها به حساب آن مهاجم منتقل شود.

سوال ۳

(a)

تخمین *Top – Down* (بر مبنای درآمد = هزینه برق) فرضیات:

قیمت هر *Bitcoin*: 82,913.37\$

پاداش هر بلاک: 3.125BTC (بعد از *halving* آوریل ۲۰۲۴)

قیمت برق: 0.006\$/kWh (بر اساس تعرفه‌ای در ایران)

تعداد بلاک‌های تولیدشده در روز: 144

محاسبات:

درآمد روزانه:

$$144 \text{ blocks/day} \times 3.125 \text{ BTC/block} \times \$82,913.37/\text{BTC} = \$37,355,000/\text{day}$$

مصرف برق روزانه (بر فرض اینکه کل درآمد صرف هزینه برق بشه):

$$\frac{\$37,355,000}{\$0.006/\text{kWh}} = 6,225,833,333 \text{ kWh/day}$$

مصرف برق سالانه:

$$6,225,833,333 \text{ kWh/day} \times 365 \text{ days} = 2,272,431,166,545 \text{ kWh/year} (\approx 2,272 \text{ TWh/year})$$

نکته مهم: در روش *Top – Down* فرض می‌کنیم که تمام درآمد حاصل از ماینینگ صرف هزینه برق می‌شود. این فرض باعث می‌شود تخمین ما کمی بالا باشد چون هزینه‌هایی مثل تجهیزات، خنک‌سازی، نگهداری، سودآوری و غیره رو در نظر نمی‌گیریم. این تخمین، به نوعی تخمین حد بالا از برق لازم برای ماینینگ بیت‌کوین است و این یعنی اگر بیشتر از این مقدار هزینه برق کنیم، صرفه اقتصادی ندارد.

(b)

تخمین *Bottom – Up* (بر اساس راندمان سخت‌افزار ماینینگ) فرضیات:*Hashrate* فعلی شبکه:

$$1,030.93, \text{EH/s} = 1,030,930,000, \text{TH/s}$$

راندمان انرژی سخت‌افزار: $21.5, \text{J/TH}$ (بر اساس مشخصات دستگاه‌هایی مثل *Bitmain S19*)

محاسبات:

مصرف توان کل شبکه:

$$1,030,930,000 \text{ TH/s} \times 21.5 \text{ J/TH} = 22,164,995,000 \text{ W} = 22.16 \text{ GW}$$

مصرف روزانه انرژی:

$$22.16 \text{ GW} \times 24 \text{ hours} = 531.84 \text{ GWh/day}$$

مصرف سالانه انرژی:

$$531.84 \text{ GWh/day} \times 365 \text{ days} = 194,126.4 \text{ GWh/year} = 194.13 \text{ TWh/year}$$

این روش بر اساس سخت‌افزار به‌روز و مشخصات فنی آن، مصرف واقعی‌تر و دقیق‌تری از انرژی شبکه بیت‌کوین ارائه می‌دهد.

(c)

مقایسه و تحلیل در روش $Top - Down$ ، مصرف سالانه برق شبکه تقریباً برابر با $2,272, TWh$ تخمین زده شده؛ در حالی که در روش $Bottom - Up$ ، این مقدار حدود $194.1, TWh$ برآورد می‌شود. درواقع حد بالا و حداکثر برق مصرفی، تقریباً ۱۰ برابر حد پایین و حداقل برق لازم برای برآورده کردن آن میزان $hashrate$ است.

این تفاوت قابل توجه به دلیل تفاوت در فرضیات بنیادی دو روش است:

روش $Top - Down$ فرض می‌کند همه درآمد ماینرها صرف برق می‌شود، که در واقعیت این‌طور نیست. هزینه‌های جانبی، سودآوری و سرمایه‌گذاری سخت‌افزاری در نظر گرفته نمی‌شوند. این روش به نوعی حد بالای لازم برای مصرف برق ماینرها است. چون بیشتر از این مقدار صرفه نداشته و کسی برای آن هزینه نخواهد کرد.

روش $Bottom - Up$ بر اساس راندمان سخت‌افزار و $hashrate$ واقعی عمل می‌کند، و بنابراین به تخمینی واقعی‌تر و محافظه‌کارانه‌تر می‌رسد. این روش درواقع حد پایین و حداقل برق لازم برای برآورده کردن محاسبات آن میزان $hashrate$ است.

توجه داشته باشید که مقدار $hashrate$ که در محاسبات استفاده شده، بر اساس مقدار $hashrate$ شبکه در آن لحظه حل تمرین بوده و ممکن است الان مقدار آن در شبکه متفاوت باشد.

نتیجه: روش $Bottom - Up$ دقیق‌ترست چون به مشخصات فنی و واقعیات بازار تکیه دارد، در حالی که روش $Top - Down$ بیشتر برای حد بالای مصرف انرژی به کار می‌رود.

سوال ۴

(a)

برای اینکه ثابت شود که تراکنش بر روی شبکه بلاک‌چین به صورت $on - chain$ هست، باید اطلاعات زیر را بدانیم:

۱. تراکنش کامل: باید اطلاعات تراکنش را به صورت کامل بدانیم تا بتوانیم با آدرس آن چک کنیم.
۲. اثبات از طریق درخت $Merkle$ (یا به انگلیسی $Merkle - proof$): باید تمام هش‌های لازم برای اعتبارسنجی تراکنش در درخت مرکل را تا ریشه درخت مرکل داشته باشیم.
۳. $Block - header$ حاوی آن تراکنش: باید بلاک هدر حاوی آن تراکنش را داشته باشیم تا درستی آن را (مثلاً با چک کردن $Merkle - root$)

۴. *Block - header* های میانی: در صورتی که میان بلاک حاوی آن تراکنش، با بلاک حال حاضر فاصله باشد، باید تمام بلاک‌هدر های میانی را داشته باشیم تا یکی پس از دیگری تناوب آن‌ها را چک کنیم (چون می‌دانیم که در هر بلاک‌هدر، هش بلاک‌هدر قبلی نیز وجود دارد) به این شکل می‌توانیم مطمئن شویم که آن تراکنش بر روی *chain* حال حاضر، موجود است.

(b)

خب اگر بخواهیم خود تراکنش را داشته باشیم، این معمولاً خودش ۲۵۰ بایت حافظه نیاز دارد. از طرفی برای *Merkle - proof* به اندازه ارتفاع درخت به هش نیاز داری و ارتفاع درخت برابر است با: $\log_2 n = 8$ و از طرفی هر هش در استاندارد *SHA - 256* ۳۲ بایت است. پس به اندازه $8 \times 32 = 256$ بایت برای اثبات درستی تراکنش در درخت مرکل نیاز است.

گفتیم که به تمام *block - header* های میانی نیاز داریم به همراه بلاک‌هدر حال حاضر. پس k تا بلاک‌هدر میانی با ۱ بلاک‌هدر حال حاضر در مجموع $k + 1$ تا بلاک‌هدر نیاز است. هر بلاک‌هدر ۸۰ بایت نیاز دارد. (درواقع داریم: نسخه (*Version*): ۴ بایت نسخه پروتکل، هش بلاک قبلی (*Previous block hash*): ۳۲ بایت لینک به بلاک والد، ریشه مرکل (*Merkle root*): ۳۲ بایت هش ریشه تراکنش‌های بلاک، زمان (*Timestamp*): ۴ بایت زمان ساخت بلاک، هدف سختی (*Difficulty target*): ۴ بایت سختی استخراج، نانس (*Nonce*): ۴ بایت پاسخ معمای ماینینگ، جمع کل: ۸۰ بایت.) پس در مجموع برای بلاک‌هدرها داریم: $80 \times (k + 1) = 80 \times 9 = 720 \text{ bytes}$ بنابراین، در مجموع تقریباً به $250 + 256 + 720 = 1226 \text{ bytes}$ نیاز داریم.

(c)

نحوه کاهش اندازه اثبات:

(ما مقدار ثابت سایز هش را با h تقریباً برابر ۳۲ بایت و مقدار ثابت سایز بلاک‌هدر را با H تقریباً برابر ۸۰ بایت به علاوه چند بایت برای پوینتر جدید نمایش می‌دهیم.)

اندازه در بهترین حالت:

بهترین حالت زمانی است که بلاک حاوی تراکنش، جدیدترین بلاک باشد، یا بلاکی باشد که اشاره‌گر هش کوچکتر سر فعلی، مستقیماً به آن اشاره کند. در این حالت، اثبات حداکثر شامل یک هدر بلاک (علاوه بر اثبات مرکل) خواهد بود. پس اندازه در بهترین حالت $O(h \log n) = O(\log n)$ بایت است.

اندازه در بدترین حالت:

بدترین حالت زمانی است که اشاره‌گرهای هش کوچکتر به طور مداوم به بلاک‌هایی اشاره کنند که فقط کمی فاصله تا بلاک هدف را کاهش می‌دهند. در یک سناریوی افراطی (اگرچه با یک تابع هش خوب بعید است)، اشاره‌گرهای هش

کوچکتر ممکن است به مسیری با طول نزدیک به k منجر شوند. k هدر به علاوه اثبات مرکب، اگر کلاینت نیاز به تأیید کل زنجیره داشته باشد. مثل قبل یدونه یدونه عقب می رود. اندازه در بدترین حالت، از اردر $O(Hk) + O(h \log n) = O(k) + O(\log n)$ بایت است.

اندازه در حالت *expected*:

در این حالت، در صورتی که هش‌ها به صورت توزیع *uniform* تولید شده باشند، به گونه‌ای می‌توان گفت که اشاره‌گر جدید، فاصله k هدر را نصف می‌کند. پس هر بار مسیر نصف می‌شود و در نتیجه به $\log k$ تا هدر نیاز داریم پس در مجموع برای حالت *expected* از اردر $O(H \log k) + O(h \log n) = O(\log k) + O(\log n)$ بایت نیاز است.

مراجع

- [1] "Signature malleability," <https://medium.com/draftkings-engineering/signature-malleability>, 2023.

Blockchain-HW2

5

a)

Yes, there is a chance that a permanent chain split can occur due to an exploit by the malicious party, even with only 10% of the hashing power. Here's why:

1. The malicious party can exploit differences in consensus rules between the client versions (0.15, 0.16, 0.17) to create blocks that are valid on some clients but not others.
2. This can lead to nodes running different versions following different chains.
3. With 40% of the network on version 0.17, if the attacker supports the chain followed by 0.17 nodes, they can combine their 10% to make 50%, allowing both chains to have substantial support.
4. If neither chain is abandoned by its supporters, the split can become permanent.

In a blockchain network like Bitcoin, a **permanent chain split (fork)** occurs when the network divides into two separate chains that continue to exist indefinitely, with neither chain being abandoned by the network. A **50-50 split** means that roughly half of the network's mining power supports one chain, while the other half supports the competing chain.

A **50-50 split** is possible if:

1. Different versions enforce different rules.
2. An attacker exploits these differences to create conflicting blocks.
3. Mining power is evenly split between the two chains (e.g., 50% vs. 50%).
4. Neither side gives up, making the fork permanent.

Example: Exploiting a Rule Change in v0.17

Assumptions:

- **v0.17 introduces a new consensus rule** (e.g., a new transaction format or block validation rule).
- **v0.15 and v0.16 do not enforce this rule** and will accept blocks that v0.17 rejects.
- **Malicious miner controls 10% of hashing power** and wants to cause a permanent split.

Step-by-Step Attack:

1. Malicious Miner Creates a "Divisive Block":

- The attacker mines a block (**Block X**) that **follows old rules (valid in v0.15/v0.16) but breaks the new rule in v0.17**.
- **v0.15 (20%) + v0.16 (30%) = 50%** accept **Block X**.
- **v0.17 (40%) rejects Block X** and continues mining on the previous block.

2. Network Splits into Two Chains:

- **Chain A (50% support):** Includes **Block X** (followed by v0.15 + v0.16 miners).
- **Chain B (40% support):** Excludes **Block X** (followed by v0.17 miners).

3. Attacker Tips the Balance to 50-50:

- The attacker **stops mining on Chain A** and **switches their 10% to Chain B**.
- Now:
 - **Chain A = 50% (v0.15 + v0.16)**.
 - **Chain B = 40% (v0.17) + 10% (attacker) = 50%**.
- Both chains now have **equal mining power (50-50)**.

4. Result: Permanent Split

- Both chains continue growing at roughly the same rate.
- Neither chain is abandoned because:
 - **Chain A** is supported by v0.15/v0.16 nodes (50%).
 - **Chain B** is supported by v0.17 + attacker (50%).
- The attacker **sustains the split indefinitely** by mining on Chain B.

Real-World Example: Bitcoin Cash Fork (2017)

While not caused by an exploit, the **Bitcoin Cash (BCH) hard fork** is an example of a chain split where:

- One group of miners (supporting larger blocks) continued on Bitcoin Cash.
- The other group (supporting SegWit) stayed on Bitcoin.
- Both chains survived because they had enough mining power and community support.

In our hypothetical scenario, the attacker **artificially creates this split** by exploiting version differences, leading to a **sustained 50-50 split** if mining power remains balanced.

b)

The given formula calculates the probability r that an attacker with **hashing power q (where $q < 0.5$)** can successfully reverse a transaction that has received **n confirmations**.

The formula is:

$$r = 1 - \sum_{m=0}^n \binom{m+n-1}{m} ((1-q)^n q^m - (1-q)^m q^n)$$

Where:

- q = Attacker's hashing power (fraction of total network power).
- n = Number of confirmations the receiver waits for.
- m = Number of blocks the attacker mines in competition with the honest chain.

So it is:

$$r = 1 - \sum_{m=0}^6 \binom{m+5}{m} ((0.9)^6 (0.1)^m - (0.9)^m (0.1)^6)$$

and that is approximately about: (by deepseek, i didn't calculate it myself)

$$\approx \frac{1}{531441} \approx 0.000188\%$$

- **Extremely low chance:** The attacker has only a **~1 in 500,000** chance of success.

c)

1. Ali's Strict Confirmation Policy

- **Condition:** Ali does **not** consider transactions as "confirmed" if:
 - The **total value of transactions in a block exceeds the block reward** (currently 3.125 BTC).
 - **Unless** they receive **144 confirmations** (~1 day, since 1 block ≈ 10 minutes).
- **Implication:** High-value transactions (e.g., >3.125 BTC) require **24 hours** to finalize.

2. Bitcoin Network's Agility

- **Assumption:** Miners/developers can detect and **patch double-spend attacks within a day**.
 - This means any exploit would be **short-lived** (≤144 confirmations).

3. Would an Attacker Target Ali?

Answer: No, it would not be logical for an attacker to double-spend against Ali. Here's why:

a) High Confirmations (144) Make Attacks Nearly Impossible

- The probability of reversing **144 confirmations** is astronomically low, even for powerful attackers.
 - For $q=0.1$ (10% attacker), the success chance is

$$\left(\frac{0.1}{0.9} \right)^{144} \approx 10^{-144}$$

- **Conclusion:** No rational attacker would attempt this.

b) Attack Would Be Detected and Patched Quickly

- Since the network can **detect and fix** double-spend attacks within a day:
 - The attacker's window of opportunity is **<144 confirmations**.
 - But Ali **only accepts high-value TXs after 144 confirmations**, so the attack would fail.

c) Economic Incentives Don't Justify the Attack

- **Cost of Attack:**
 - To sustain a 10% hashrate for 24 hours, the attacker must spend heavily on mining.
 - If caught, they risk being **blacklisted** or forked out.
- **Potential Reward:**
 - Only transactions **>3.125 BTC** are at risk, but Ali's policy nullifies this.
 - The attacker gains **nothing** because Ali's 144-confirmation rule ensures safety.

d) Alternative Targets Are More Profitable

- Attackers would prefer **less cautious users** who accept fewer confirmations.
- Ali's policy makes him a **terrible target** compared to others.

Ali's strict policy effectively **neutralizes double-spend risk**, making him an unattractive target.

6

a) **scriptPubKey** (locking script):

```
OP_OVER OP_OVER OP_EQUAL OP_NOT OP_VERIFY OP_RIPEMD160 OP_SWAP OP  
_RIPEMD160 OP_EQUAL
```

b) **scriptSig** (unlocking script):

```
<input2> <input1>
```

(Where `input1` and `input2` are two distinct messages such that `RIPEMD160(input1) == RIPEMD160(input2)` .)

Explanation:

1. **Initial Stack** (after `scriptSig` provides `input1` and `input2`):

Copy

```
[input1, input2]
```

2. **OP_OVER** :
 - Copies `input1` to the top.
 - Stack: `[input1, input2, input1]`
3. **OP_OVER** :
 - Copies `input2` to the top.
 - Stack: `[input1, input2, input1, input2]`
4. **OP_EQUAL** :
 - Checks if `input1 == input2` .
 - Pushes `1` if equal, `0` otherwise.
 - Stack: `[input1, input2, (input1 == input2 ? 1 : 0)]`
5. **OP_NOT** :
 - Inverts the result (`1` → `0` , `0` → `1`).
 - Stack: `[input1, input2, (input1 != input2 ? 1 : 0)]`
6. **OP_VERIFY** :
 - Fails if `input1 == input2` (top item is `0`).
 - Otherwise, continues with stack: `[input1, input2]`
7. **OP_RIPEMD160** :

- Hashes `input2` → `hash2` .
 - Stack: `[input1, hash2]`
8. **OP_SWAP** :
- Swaps `input1` and `hash2` .
 - Stack: `[hash2, input1]`
9. **OP_RIPEMD160** :
- Hashes `input1` → `hash1` .
 - Stack: `[hash2, hash1]`
10. **OP_EQUAL** :
- Checks if `hash1 == hash2` .
 - Pushes `1` (valid) or `0` (invalid).
- If both conditions pass (`input1 != input2` and `hash1 == hash2`), the script is valid, and the prize is claimed.

7

To determine whether these features require a **soft fork** or a **hard fork**, we need to recall the key difference:

- **Soft Fork**: Backward-compatible. Stricter rules — old nodes still see blocks as valid, even if they don't enforce the new rules.
- **Hard Fork**: Not backward-compatible. Looser or entirely new rules — old nodes will reject new blocks or transactions.

(a) Adding a new OP_SHA3 script instruction

- **Answer: Hard Fork**
- **Reason**: Adding a new script instruction (e.g., OP_SHA3) introduces new functionality that **old nodes don't recognize**. They would reject transactions using this opcode as invalid.

(b) Disabling the OP_SHA1 instruction

- **Answer: Soft Fork**
 - **Reason:** Disabling an existing opcode **tightens** the consensus rules — scripts that used to be valid become invalid. But since old nodes already accepted those scripts, they will still accept blocks that no longer include them. This is a soft fork.
-

(c) A requirement that each miner include a Merkle root of unspent transaction outputs (UTXOs) in each block

- **Answer: Hard Fork**
 - **Reason:** This changes the **block structure** (adds a new field). Old nodes wouldn't recognize or understand the new block format, so they would reject the blocks. That's a hard fork.
-

(d) A requirement that all transactions have their outputs sorted by value in ascending order

- **Answer: Soft Fork**
 - **Reason:** This adds a **new validation rule** (output ordering), making blocks that don't follow the rule invalid under new consensus, but still valid to old nodes. That makes it a soft fork.
-

Summary Table

Feature	Fork Type	Reason
(a) Add OP_SHA3	Hard Fork	Adds new opcode — not recognized by old nodes
(b) Disable OP_SHA1	Soft Fork	Makes some scripts invalid — tighter rules
(c) UTXO Merkle root in blocks	Hard Fork	Changes block format — incompatible with old nodes
(d) Sorted outputs by value	Soft Fork	Adds stricter transaction structure rule

8

(a)

How many blocks were abandoned due to the version 0.8 bug?

In March 2013, a significant hard fork occurred due to an incompatibility between Bitcoin Core versions 0.7 and 0.8. Version 0.8 introduced a change in the database engine from BerkeleyDB to LevelDB. This change led to a situation where version 0.8 nodes accepted a large block that version 0.7 nodes rejected, causing a split in the blockchain. To resolve the issue, miners downgraded to version 0.7, resulting in the abandonment of 24 blocks that had been mined on the version 0.8 chain. You can check it in <https://blog.citp.princeton.edu/analyzing-the-2013-bitcoin-fork-centralized-decision-making>.

(b)

How many blocks were orphaned due to the addition of pay-to-script-hash?

The pay-to-script-hash (P2SH) feature was introduced as a soft fork through BIP 16 in 2012. During its activation, there was a period when some miners had not yet upgraded to support P2SH, leading them to mine blocks that were later rejected by the majority of the network. While the exact number of orphaned blocks isn't specified in the available sources, it's noted that multiple blocks containing certain transactions were orphaned because they didn't comply with the new P2SH rules

- In the case of the **P2SH soft fork**, after it activated with only 55% miner support, **the remaining 45% continued to mine blocks that were invalid under the new rules** (i.e. blocks with non-standard transactions that violated P2SH rules).
- These **invalid blocks were not just orphaned** — they were **rejected entirely** by upgraded nodes. So while some were technically "orphaned," many didn't even make it into the blockchain at all because they failed validation.
- After activation, problems were caused as the remaining 45% of miners produced invalid blocks **for several months after the soft-fork**.

So one question can be:

"Isn't it a soft fork? Shouldn't that mean all upgraded nodes reject invalid blocks, and no orphaning happens? Were any blocks orphaned?"

- Technically: **Yes**, some blocks were orphaned — but **only among un-upgraded nodes**.
- On the **main (upgraded) chain**, any blocks violating P2SH rules were **not just orphaned — they were *invalid*** and rejected before ever becoming part of the chain.

✓ Conclusion:

Yes, some orphaned blocks occurred among un-upgraded nodes, but not on the main chain. On the main network, those blocks were invalid, not orphaned.

You can check it in: <https://blog.bitmex.com/bitcoins-consensus-forks/>

I read somewhere that:

Only **one block** was orphaned due to P2SH's introduction, caused by a mining pool's failure to properly implement the new rules. After this, P2SH was widely adopted without further issues.

I even see that someone says there were 3 blocks orphan in that time. but I think the best answers is the first answer. they were not orphans. they were invalid blocks. (they maybe count as orphaned blocks only for the 45% old miners that mined for several months)

(c)

What is "Safe Mode"?

"Safe mode" is a protective state Bitcoin clients can enter when they detect something suspicious — like a fork. In safe mode, certain features (like sending coins) may be disabled to protect users until the situation is resolved.

1. Chain Reorganizations (Reorgs)

- A reorg happens when the client replaces part of its chain with a longer one from the network.
- A large or frequent reorg (e.g., 3+ blocks) is unusual and suspicious.
- Heuristic:

"If I see a chain reorg deeper than X blocks, I might be on the wrong chain — enter safe mode."

2. Blocks with Unexpected Rules

- If a node receives a block that is *valid according to proof-of-work*, but contains transactions it considers invalid (due to new soft fork rules), this signals a **fork in consensus rules**.
- Heuristic:

"If I receive valid PoW blocks that break my transaction rules — there's likely a fork."

3. Inconsistent Block Version Numbers

- Forks are often signaled with a new version number.
- Seeing a new version from some miners but not others can be a red flag.
- Heuristic:

"If new block versions appear inconsistently — maybe some miners are on a different protocol."

4. Sudden Spike in Rejected Blocks

- If many blocks are being rejected (i.e., failing validation), it might mean you're on a minority fork or others are.
- Heuristic:

"If I start rejecting a lot of blocks that others seem to accept — something's wrong."

5. Peer Disagreement

- If a node's peers are all following a different chain (e.g., your best chain is not the one most peers are on), that's a major warning.
- Heuristic:

"If most of my peers are on a different chain tip than me — don't trust my current chain."

9

(a) Low-Cost Method (Transaction Fee Only)

To write your name into the blockchain without spending more than a standard transaction fee, you can embed it in the **OP_RETURN** field of a transaction. This allows you to store arbitrary data (up to 80 bytes) in a provably unspendable output.

Steps:

1. **Create a Bitcoin transaction** with at least one input (funds you control) and one output (can be sent back to yourself or another address).
2. **Add an **OP_RETURN** output** containing your name encoded in hex or ASCII.
 - Example: If your name is "Alice", you could encode it as **416c696365** (hex for "Alice").
3. **Broadcast the transaction** to the Bitcoin network.

Pros:

- Very cheap (just the transaction fee).
- Your name will be permanently stored in the blockchain.

Cons:

- Most block explorers won't display it in a human-readable way unless you decode it manually.

(b) Human-Readable Method (Visible in Block Explorers)

If you want your name to appear visibly in common block explorers (like [Blockstream Explorer](#) or [Blockchain.com](#)), you can use **"Coinbase" transactions** (miner-created transactions) or **"fake" Bitcoin addresses** that spell your name in Base58.

Method 1: Fake Address with Your Name

Bitcoin addresses are Base58-encoded, so you can try to generate a vanity address that starts with your name (e.g., `1Alice...`). However, this is computationally expensive unless your name is very short.

- Use tools like **Vanitygen** or **Bitcoin Vanity Address Generator**.
- If your name is long, this might take a very long time (or be impossible).

Method 2: Embed in a Miner's Coinbase Transaction

If you mine a block (or convince a miner to help), you can write arbitrary data in the **coinbase parameter** (the first transaction in a block). Some miners allow you to pay them to include custom text.

- Example: The coinbase scriptSig could contain `"Alice was here"`.
- This is expensive (you'd need to mine a block or pay a miner).

Method 3: Use a Large **OP_RETURN** (Non-Standard)

Some block explorers display large **OP_RETURN** data in a readable format. You could encode your name in a way that some explorers interpret it as text.

Summary

Method	Cost	Visibility	Difficulty
(a) OP_RETURN	Cheap (tx fee)	Needs decoding	Easy
(b1) Vanity Address	Free (but slow)	Visible as address	Hard (for long names)
(b2) Coinbase Text	Expensive (mining cost)	Visible in block explorers	Very hard unless you mine

(b3) Non-standard OP_RETURN	Moderate	Depends on explorer	Medium
------------------------------------	----------	---------------------	--------

(b1): About vanity addresses we should know that:

| The longer the prefix, the longer it takes (exponentially harder).

I mean:

- "Alice" (5 chars) could take **hours to days** on a CPU.
- "AL" (2 chars) takes seconds.

(b3): Method 3: Using a Non-Standard **OP_RETURN** for Human-Readable Text

Key Idea:

Some block explorers (like [Blockstream Explorer](#)) automatically decode **OP_RETURN** data if it looks like **ASCII text**. If you encode your name in a way that resembles plain text (rather than random hex), these explorers may display it directly.

Example:

1. Suppose your name is **"Alice"**.
2. In hex, "Alice" is **0x416c696365**.
3. Normally, you'd push this into **OP_RETURN** as:

```
OP_RETURN 416c696365
```

- Most explorers will just show the hex (**416c696365**).
4. But if you structure the data to **look like ASCII**, some explorers might display it as text.

How to Do It:

Instead of just pushing raw hex, you can encode the data in a way that hints to the explorer that it's text. Some explorers check if the data is **UTF-8** or **ASCII** and display it accordingly.

- **Example Transaction:**

```
OP_RETURN "Alice"
```

- Some wallets allow you to input text directly (they convert it to hex).
- If the explorer sees `0x416c696365` and detects it's ASCII, it may render "Alice".

10

Key Idea:

Bitcoin payments are **pushed** by the sender (Alice) rather than **pulled** by the payment service. That means Alice initiates the payment directly on the Bitcoin network.

So the payment service can't just assume Alice paid just because she clicked "Send." Instead, it needs to verify that her transaction actually appeared on the blockchain.

How can the payment service verify Alice paid?

1. Alice is assigned a unique receiving address:

- When Alice initiates the payment, the payment service generates a **unique Bitcoin address** just for her.

Each user (like Alice) gets a **unique Bitcoin address**, and the service keeps track of which address was assigned to which user/session. Then:

- If a payment appears on the blockchain to that address → it means that user has paid.
- If no money shows up → no payment was made.

This method scales well, even with thousands of users, because generating addresses is cheap and fast, and it's easy to monitor the blockchain for incoming payments to those addresses.

- Alice sends her payment to this address.

2. Service monitors the blockchain (or mempool):

- The service watches the blockchain (and optionally the mempool) for a transaction:
 - Whose **output** is to that unique address.
 - Whose **amount** matches (or is close to) the expected amount.
 - Which comes **within a certain time window** from Alice's login/session.

3. Additional checks:

- The payment can be linked to Alice via:
 - The time of payment.
 - The amount paid.
 - Metadata she provides (like an order ID or invoice number).
- Once the transaction is **confirmed** (typically 1-6 confirmations), the payment is considered valid.

Example: Alice Pays Using a Bitcoin Payment Service

Step 1: Alice logs in and wants to pay

- Alice visits the website to buy a product for **0.01 BTC**.
- The payment service creates a **unique Bitcoin address** just for her, e.g.:

```
1AlicePaymentAddressXYZ123
```

Step 2: Alice is shown payment details

- The website shows:
 - Amount: **0.01 BTC**
 - Address: **1AlicePaymentAddressXYZ123**
 - QR code or link to pay

Step 3: Alice pays from her wallet

- Alice opens her Bitcoin wallet and sends **0.01 BTC** to:

1AlicePaymentAddressXYZ123

Step 4: Service monitors the blockchain

- The payment service is watching for incoming transactions to:

1AlicePaymentAddressXYZ123

- It sees a new transaction:
 - To: **1AlicePaymentAddressXYZ123**
 - Amount: **0.01 BTC**
 - Time: within a few minutes of Alice's session

Step 5: Payment is confirmed

- Once the transaction gets **1 or more confirmations**, the service marks Alice's order as **paid**.

✅ Done:

- The service knows **for sure** that Alice paid — because that unique address was tied only to her.

You should notice that:

we **won't run out of Bitcoin addresses**, even if we never reuse them.

◆ Why not?

Bitcoin addresses are based on **public key cryptography**. Specifically:

- Each address comes from a **public key**, which comes from a **private key**.
- The total number of possible private keys is:

$$2^{256} \approx 1.16 \times 10^{77}$$

That's more than the number of atoms in the universe.

So even if you generate **a new address for every coffee you buy**, humanity **will never run out** — not even close.

11

a)

How TransparentExchange Might Cheat the Proof of Reserve

The current Proof of Reserve mechanism involves:

1. Publishing a list of addresses with a total balance of 500,000 BTC.
2. Signing the statement "*TransparentExchange controls at least 500,000 BTC*" with each corresponding private key.
3. Presenting these signatures as proof.

Potential ways the exchange could cheat:

1. Borrowing BTC Temporarily (Proof Inflation Attack)

- The exchange could temporarily borrow BTC from other entities (e.g., other exchanges, custodians, or large holders) to inflate their balance at the time of proof generation.
- After the proof is published, they return the borrowed BTC, meaning they never truly controlled the full amount.

2. Reusing the Same BTC Across Multiple Addresses (Double-Counting Attack)

- If the exchange moves the same BTC between different addresses under its control, it could create the illusion of multiple independent holdings.
- For example, they could sign with multiple keys but actually control the same coins moved around in a short time frame.

3. Using Unspent Outputs from Past Transactions (Historical Balance Attack)

- The exchange could present addresses that once held BTC but no longer do (e.g., by signing an old statement or reusing signatures from a time when they had more BTC).

- The blockchain would show that those addresses are now empty or underfunded.

4. Colluding with Miners or Using Unconfirmed Transactions (Temporal Attack)

- The exchange could collude with miners to show unconfirmed transactions that temporarily inflate their balance.
- Alternatively, they could use pending transactions to make it seem like they control more BTC than they actually do.

5. Hiding Liabilities (Selective Address Disclosure)

- The exchange might omit addresses that have large liabilities (e.g., user deposits they owe but haven't secured with actual BTC).
- This makes the *Proof of Reserve* meaningless without a corresponding *Proof of Liability*.

Example Scenario: Exchange Temporarily Borrows BTC to Fake Proof of Reserves

Let's say **TransparentExchange** is insolvent—it only holds **300,000 BTC** but claims to have **500,000 BTC** in its Proof of Reserve. To trick users, it borrows BTC to inflate its balance temporarily.

Step-by-Step Cheating Process

1. Before the Audit (Real Situation)

- **TransparentExchange's actual reserves:** 300,000 BTC (in its own wallets).
- **User deposits (liabilities):** 500,000 BTC (meaning it owes users more than it has).
- **It's insolvent** (a 200,000 BTC shortfall).

2. Borrowing BTC from a Collaborator

- The exchange contacts **BigWhale Custodian**, a large Bitcoin holder, and asks to "rent" **200,000 BTC** for a short time.
- They agree on terms (possibly a fee or future favor).
- **BigWhale sends 200,000 BTC** to addresses controlled by TransparentExchange.

3. Generating the Fake Proof of Reserve

- Now, TransparentExchange temporarily has:
 - **300,000 BTC (original) + 200,000 BTC (borrowed) = 500,000 BTC.**
- It publishes a list of addresses holding **500,000 BTC** and signs:
 - *"TransparentExchange controls at least 500,000 BTC"* with each private key.
- Users and auditors see:
 - The signatures are valid (proving control of the keys).
 - The blockchain shows the addresses **do** hold 500,000 BTC at that moment.

4. Returning the Borrowed BTC (After the Audit)

- A few hours/days later, TransparentExchange sends back the **200,000 BTC** to BigWhale.
- Now, its real reserves are back to **300,000 BTC**, but users still think it has **500,000 BTC** based on the "proof."

5. The Deception is Complete

- Users continue depositing, thinking the exchange is fully backed.
- In reality, **it's still insolvent**—if everyone tried to withdraw, it couldn't cover all requests.

How to Prevent Cheating

To create a robust Proof of Reserve system that prevents exchanges from faking their BTC holdings, we need a multi-layered approach that combines cryptographic proofs, transparency mechanisms, and independent verification. Here's the consolidated solution:

1. Time-Locked Verification

- **Require timestamped blockchain proofs**

The exchange must sign:

"We control at least 500,000 BTC as of block #[latest_block_hash]"

- Forces verification against a specific blockchain state
- Prevents temporary BTC borrowing attacks

2. Comprehensive Address Disclosure

- **Publish a Merkle tree of all reserve addresses**
 - Includes both hot and cold wallets
 - Users can verify their funds are included
 - Prevents hiding addresses or selectively reporting balances

3. On-Chain Validation

- **Require cryptographic proof of UTXOs**
 - Provide Merkle proofs showing addresses contain the claimed BTC
 - Verify against the referenced block hash
 - Prevents using empty or recently-funded addresses

4. Prevention of Double-Counting

- **Proof of non-overlapping UTXOs**
 - Demonstrate no BTC is counted in multiple addresses
 - Can use cryptographic accumulators or range proofs

5. Combined Proof of Liability

- **Mandatory pairing with liability proof**
 - Merkle tree of all user balances (sorted to prevent omission)
 - Must show: Total Reserves $\geq$ Total Liabilities
 - Prevents insolvency hiding

6. Independent Verification

- **Third-party audits**
 - Regular surprise audits by trusted firms
 - Checks for address consistency over time
- **Zero-knowledge options**
 - zk-SNARKs can prove solvency without revealing addresses

7. Monitoring Mechanisms

- **Track address activity patterns**
 - Look for suspicious inflows/outflows around proof times
 - Compare with historical reserve patterns
-

Real-World Lessons

Historical exchange failures show why these measures matter:

- **Mt. Gox:** Lack of proper reserve auditing allowed hidden insolvency
 - **QuadrigaCX:** No address transparency enabled fund misrepresentation
 - **FTX:** Absence of real liability proofs concealed shortfalls
-

Conclusion

This trick relies on **temporarily inflating balances** to fool users. Without **real-time, on-chain verification** and **liability checks**, exchanges can fake solvency. Always verify reserves independently!

b)

Why Fake Users Can't Be Added: Cryptographic Safeguards

Users can detect fraudulent activity through **two key verification mechanisms**:

1. Value Consistency Check

When verifying their Merkle proof, users confirm that:

- Every parent node's value **exactly equals** the sum of its children's values
(`parent.value = left_child.value + right_child.value`).
- No node contains a **negative value** (which would corrupt the tree's accounting).
- The parent's value is **never less than either child's value** (preventing balance inflation/deflation tricks).

If the exchange tries to insert fake negative balances:

- The math **wouldn't add up** when users verify their paths.
- To compensate, the exchange would need to artificially inflate other nodes, which would be immediately detectable.

2. **Cryptographic Hash Integrity**CopyDownload

Each node's ID is generated using a cryptographic hash function:

```
node.id = Hash(left_child.id || right_child.id || node.value)
```

This creates an **unbreakable chain of custody** because:

- Changing any value (like adding a fake user) would **alter all upstream hashes**, changing the Merkle root.
- Users would instantly notice the mismatch when comparing their proof against the published root hash.
- Reverse-engineering valid hashes is **computationally impossible** (like trying to unmix a smoothie into original ingredients).

c)

Attack on Proof of Liability & Fix

Attack: Omitting Users (Partial Tree Inclusion)

- **How it works:**
 - The exchange builds a **Merkle tree but excludes some users** (e.g., those with small balances or those who don't check their accounts).
 - Example:
 - Real deposits: 1,000 BTC (100 users).
 - Exchange only includes 90 users (900 BTC) in the tree.
 - Publishes root hash for 900 BTC, hiding the 100 BTC shortfall.
 - **Why it works:**

- Users who *are* included get valid proofs and don't suspect anything.
- Excluded users might not check (or the exchange ignores their requests).

Fix: Commit to All Users via Sorted Merkle Tree

1. **Require a sorted list of all user IDs** (e.g., alphabetical usernames). it makes that people with different balances, distribute randomly in the sorted list of users. (If we sorted based on the balances, the exchange could ignore the last users with the least balances.)
2. **Build the Merkle tree over the full sorted list**, with **0 BTC** for omitted users.
3. **Publish the full leaf count** (e.g., 100 leaves, even if some have **0**).
 - Now, if the exchange omits a user:
 - Their spot in the tree shows **0 BTC** (obvious fraud).
 - Users can verify no one is missing by checking sort order and leaf count.

Key Improvement to the Fix

Add **two additional safeguards**:

1. **Publish total user count** (e.g., "We have 100,000 customers")
 - Makes omissions mathematically detectable
2. **Require random audits**:
 - Select random users to verify inclusion
 - Probability theory ensures cheating will be caught
- Any user can check:
 - "Does the tree size match the total number of users?"
 - "Is my ID in the correct sorted position?"
- If **User1000** checks and finds their ID is **missing or has 0 BTC** , they'll know the exchange is lying
- It can be audited by some trusted third parties

d)

Privacy Leakage

Yes, the current scheme **leaks partial information** about other users:

- During a Merkle inclusion proof, a user sees **sibling nodes** along their path to the root (Figure 4.6).
 - Example: If Alice requests her proof, she might learn that her sibling node (Bob) has a balance of 10 BTC, and their parent's sibling subtree sums to 200 BTC.

Fix: Hide Sibling Values

To eliminate leaks:

1. Zero-Knowledge Proofs (ZKPs):

- Use ZKPs (e.g., zk-SNARKs) to prove that:
 - Each internal node's value equals the sum of its children.
 - No negative values exist.
- This hides all balances except the user's own.

2. Optimized Solutions:

- Protocols like **Provisions** (mentioned in the book: Bitcoin and Cryptocurrency Technologies by Arvind Narayanan) achieve this using advanced cryptography without revealing totals or individual balances.

3. Use Pedersen Commitments Instead of Plain Values:

- Instead of publishing actual balances or sums, use **cryptographic commitments** (like Pedersen commitments) to the balances.
- Pedersen commitments are *additively homomorphic*, meaning the internal nodes can still compute correct sums **without revealing the actual values**.
- They are also **hiding**: no one can tell what value is inside unless you open it.

12

(a) What determines whose block will end up on the consensus branch?

In blockchain networks like Bitcoin, when two miners (e.g., Minnie and Mynie) find valid blocks nearly simultaneously, a temporary fork occurs. The network resolves this by adopting the chain that becomes longer first. Miners continue to build on the block they received first, and eventually, one chain outpaces the other. The shorter chain's block becomes orphaned, and its transactions return to the mempool. This process ensures consensus across the network.

(b) What factors affect the rate of orphan blocks? Can you derive a formula for the rate based on these parameters?

Several factors influence the orphan block rate:

- **Network Latency:** Delays in block propagation can lead to simultaneous block discoveries.
- **Block Size:** Larger blocks take longer to propagate, increasing the chance of forks.[rostra.ai](#)
- **Block Time:** Shorter intervals between blocks raise the probability of concurrent discoveries.
- **Hash Rate Distribution:** A more decentralized network can experience higher orphan rates due to varied propagation times.

A simplified formula to estimate the orphan rate is:

Orphan Rate $\approx$ Propagation Delay / Block Time

For instance, with a propagation delay of 5 seconds and a block time of 600 seconds (10 minutes), the orphan rate would be approximately 0.0083 or 0.83%.[rostra.ai+1Bitcoin Stack Exchange+1](#)

(c) Research online for resources that measure this rate on the Bitcoin network.

Empirical studies have observed varying orphan rates over time. With optimized protocols like Compact Blocks and FIBRE, the orphan rate has been reduced to approximately 0.1–0.2% . Additionally, the Bitcoin Stack Exchange discusses factors affecting orphan block rates, noting that network latency and block propagation speed are significant contributors .[rostra.ai+1Bitcoin Stack Exchange+1btcmining.academy+3Bitcoin Stack Exchange+3rostra.ai+3](#)

Formula Recap

$$P_{\text{orphan}} = 1 - (1 - P_b)^{\int_0^{\infty} (1 - f(t)) dt}$$

(from the paper <https://sites.cs.ucsb.edu/~rich/class/cs293b-cloud/papers/bitcoin-delay>)

Where:

- (since Bitcoin aims for a new block every 10 minutes, or 600 seconds),

$$P_b = 1600 \approx 0.001667 P_b = \frac{1}{600} \approx 0.001667 P_b = 6001 \approx 0.001667$$

- $f(t)$: fraction of nodes that have received the block at time t ,
- expected *exposure time*, i.e., how long some nodes are still unaware of the new block:

$$\int_0^{\infty} (1 - f(t)) dt$$

Simplified Example Using an Exponential Model

Assume block propagation follows an **exponential distribution** (common for modeling delays in networked systems):

$$f(t) = 1 - e^{-\lambda t}$$

Then:

$$1 - f(t) = e^{-\lambda t}$$

So:

$$\int_0^{\infty} (1 - f(t)) dt = \int_0^{\infty} e^{-\lambda t} dt = \frac{1}{\lambda}$$

Let's suppose:

- **Average propagation delay** is about **12 seconds**, meaning:

$$\lambda = \frac{1}{12} \Rightarrow \int_0^{\infty} (1 - f(t)) dt = 12$$

Plug Into the Formula

$$P_{\text{orphan}} = 1 - (1 - 0.001667)^{12}$$

Let's compute:

$$(1 - 0.001667)^{12} \approx 0.9801 \Rightarrow P_{\text{orphan}} \approx 1 - 0.9801 = 0.0199$$

Result

The **estimated orphan block rate is ~1.99%**, based on:

- 12-second average propagation delay,
- 10-minute block interval.

(d) If Mynie hears about Minnie's block just before she's about to discover hers, does that mean she wasted her effort?

Yes, if Mynie receives Minnie's block before completing her own, she will discard her current work and start mining on top of Minnie's block. This means the computational effort expended on her nearly completed block does not contribute to the main chain and is effectively wasted.

(e) Do all miners have their blocks orphaned at the same rate, or are some miners affected disproportionately?

Not all miners experience orphaning equally. Factors influencing this include: rostra.ai

- **Geographical Location:** Miners closer to the majority of the network may have faster propagation times.
- **Network Infrastructure:** Miners with better connectivity can disseminate their blocks more quickly.
- **Mining Pool Size:** Larger pools may have advantages in block propagation and validation. [Bitcoin Stack Exchange](https://BitcoinStackExchange.com)+1btcmining.academy+1

Consequently, miners with optimal network conditions and locations are less likely to have their blocks orphaned compared to those in less favorable situations.

13

(a)

Why Use the Poisson Process?

The **Bitcoin network** produces blocks at random intervals, but **on average** one block is found every **10 minutes**.

This kind of situation—**random events happening independently over time with a constant average rate**—is exactly what a **Poisson process** models.

In a Poisson process:

- The number of events in a time interval follows the **Poisson distribution**.
- The **time between events** follows the **Exponential distribution**.
- The **time until the k-th event** follows the **Erlang (or Gamma)** distribution (if k is an integer).

Probability that a block will be found in the next 10 minutes

Let's define:

- The block-rate

$$\lambda = \frac{1}{10}bpm = blockrate$$

- The exponential distribution is given by:

$$P(T \leq t) = 1 - e^{-\lambda t}$$

- For t=10 minutes:

$$P(T \leq 10) = 1 - e^{-\frac{1}{10} \cdot 10} = 1 - e^{-1} \approx 1 - 0.3679 \approx \boxed{0.6321}$$

✓ **Answer (a):** The probability is approximately **63.2%**.

(b)

Find x such that there is a 99% chance of seeing 6 blocks in x minutes

This is about finding how much time x is needed for **6 events** (blocks) to occur with **99% confidence**.

Since the **sum of k exponential random variables** is a **Gamma distribution**, for integer k it's an **Erlang distribution**.

Why Erlang?

We want the **time until 6 blocks** are found. That's the **sum of 6 independent exponential waiting times** (for each block). This total time follows the **Erlang distribution**.

The Erlang distribution is a special case of the **Gamma distribution**, where the shape parameter k is an integer (number of events).

Parameters:

- $k=6$
- $\lambda=1/10$
- Want: $P(T \leq x) = 0.99$

$$\text{where : } T \sim \text{Erlang}(k = 6, \lambda = \frac{1}{10})$$

So, find the **99th percentile** of the Erlang distribution with those parameters.

We can use the inverse CDF of the Erlang (or Gamma) distribution:

Let's use the formula:

$$x = \text{GammaInv}(0.99; \text{shape} = 6, \text{rate} = \frac{1}{10}) = \text{GammaInv}(0.99; \alpha = 6, \beta = 10)$$

This is a known quantile and numerically:

$$x \approx \boxed{111.5} \text{ minutes}$$

✅ **Answer (b):** Taghi should choose **x=111.5 minutes** to be 99% confident that **6 blocks** will be found in that time.